

Chapter Three: Streams and File I/O

Streams

- ***Stream***: an object that either delivers data to its destination (screen, file, etc.) or that takes data from a source (keyboard, file, etc.)
 - it acts as a buffer between the data source and destination
- ***Input stream***: a stream that provides input to a program
 - `System.in` is an input stream
- ***Output stream***: a stream that accepts output from a program
 - `System.out` is an output stream
- A stream connects a program to an I/O object
 - `System.out` connects a program to the screen
 - `System.in` connects a program to the keyboard

Binary Versus Text Files

- All data and programs are ultimately just zeros and ones
 - each digit can have one of two values, hence *binary*
 - *bit* is one binary digit
 - *byte* is a group of eight bits
- *Text files*: the bits represent printable characters
 - one byte per character for ASCII, the most common code
 - for example, Java source files are text files
 - so is any file created with a "text editor"
- *Binary files*: the bits represent other types of encoded information, such as executable instructions or numeric data
 - these files are easily read by the computer but not humans
 - they are *not* "printable" files
 - actually, you *can* print them, but they will be unintelligible
 - "printable" means "easily readable by humans when printed"

Java: Text Versus Binary Files

- Text files are more readable by humans
- Binary files are more efficient
 - computers read and write binary files more easily than text
- Java binary files are portable
 - they can be used by Java on different machines
 - Reading and writing binary files is normally done by a program
 - text files are used only to communicate with humans

Java Text Files

- Source files
- Occasionally input files
- Occasionally output files

Java Binary Files

- Executable files (created by compiling source files)
- Usually input files
- Usually output files

Text Files vs. Binary Files

- Number: 127 (decimal)
 - Text file
 - Three bytes: “1”, “2”, “7”
 - ASCII (decimal): 49, 50, 55
 - ASCII (octal): 61, 62, 67
 - ASCII (binary): 00110001, 00110010, 00110111
 - Binary file:
 - One byte (byte): 01111110
 - Two bytes (short): 00000000 01111110
 - Four bytes (int): 00000000 00000000 00000000 01111110

Text File I/O

- Important classes for text file **output** (to the file)
 - **PrintWriter**
 - **FileOutputStream** [or **FileWriter**]
- Important classes for text file **input** (from the file):
 - **BufferedReader**
 - **FileReader**
- **FileOutputStream** and **FileReader** take file names as arguments.
- **PrintWriter** and **BufferedReader** provide useful methods for easier writing and reading.
- Usually need a combination of two classes
- To use these classes your program needs a line like the following:

```
import java.io.*;
```

Buffering

- **Not buffered:** each byte is read/written from/to disk as soon as possible
 - “little” delay for each byte
 - A disk operation per byte---higher overhead
- **Buffered:** reading/writing in “chunks”
 - Some delay for some bytes
 - Assume 16-byte buffers
 - Reading: access the first 4 bytes, need to wait for all 16 bytes are read from disk to memory
 - Writing: save the first 4 bytes, need to wait for all 16 bytes before writing from memory to disk
 - A disk operation per a buffer of bytes---lower overhead

- **Every File Has Two Names**

1. The stream name used by Java

- outputStream in the example

2. The name used by the operating system

- out.txt in the example

Text File Output

- To open a text file for output: connect a text file to a stream for writing

```
PrintWriter outputStream = new PrintWriter(new  
    FileOutputStream("out.txt"));
```

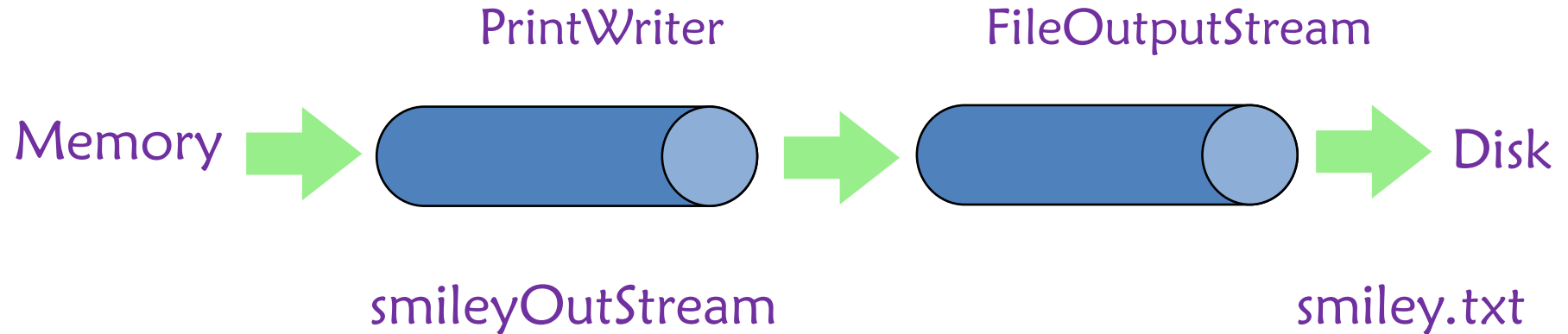
- Similar to the long way:

```
FileOutputStream s = new  
FileOutputStream("out.txt");
```

```
PrintWriter outputStream = new PrintWriter(s);
```

- Goal: create a `PrintWriter` object
 - which uses `FileOutputStream` to open a text file
- `FileOutputStream` “connects” `PrintWriter` to a text file.

Output File Streams



```
PrintWriter smileyOutputStream = new PrintWriter(new  
    FileOutputStream("smiley.txt") );
```

Methods for PrintWriter

- Similar to methods for System.out
 - println
 - `outputStream.println(count + " " + line);`
 - print
 - format
 - flush: write buffered output to disk
 - close: close the PrintWriter stream (and file)

TextFileOutputDemo - Part 1

```
public static void main(String[] args)
{
    PrintWriter outputStream = null;
    Scanner keyboard=null;
    try
    {
        outputStream = PrintWriter(new FileOutputStream("out.txt"));

        keyboard = new Scanner(System.in);
    }
    catch(FileNotFoundException e)
    {
        System.out.println("Error opening the file out.txt. “
            + e.getMessage());
        System.exit(0);
    }
}
```



TextFileOutputDemo - Part 2

```
System.out.println("Enter three lines of text:");
String line = null;
int count;
    for (count = 1; count <= 3; count++)
    {
        line = keyboard.nextLine();
        outputStream.println(count + " " + line);
    }
    outputStream.close();
    System.out.println("... written to out.txt.");
}
```

Writing to the file

Closing the file

The `println` method is used with two different streams: `outputStream` and `System.out`

Overwriting a File

- Opening an output file creates an empty file
- Opening an output file creates a new file if it does not already exist
- Opening an output file that already exists eliminates the old file and creates a new, empty one
 - Data in the original file is lost

Appending to a Text File

- To add/append to a file instead of replacing it, use a different constructor for **FileOutputStream**:
 `outputStream = new PrintWriter(new
 FileOutputStream("out.txt", true));`
- Second parameter: append to the end of the file if it exists?
- Sample code for letting user tell whether to replace or append:

```
System.out.println("A for append or N for new file:");  
char ans = keyboard.next().charAt(0);  
boolean append = (ans == 'A' || ans == 'a');  
outputStream = new PrintWriter(  
    new FileOutputStream("out.txt", append));
```

Closing a File

- An output file should be closed when you are done writing to it (and an input file should be closed when you are done reading from it).
- Use the close method of the class PrintWriter (BufferedReader also has a close method).
- For example, to close the file opened in the previous example:

outputStream.close();

- If a program ends normally it will close any files that are open.

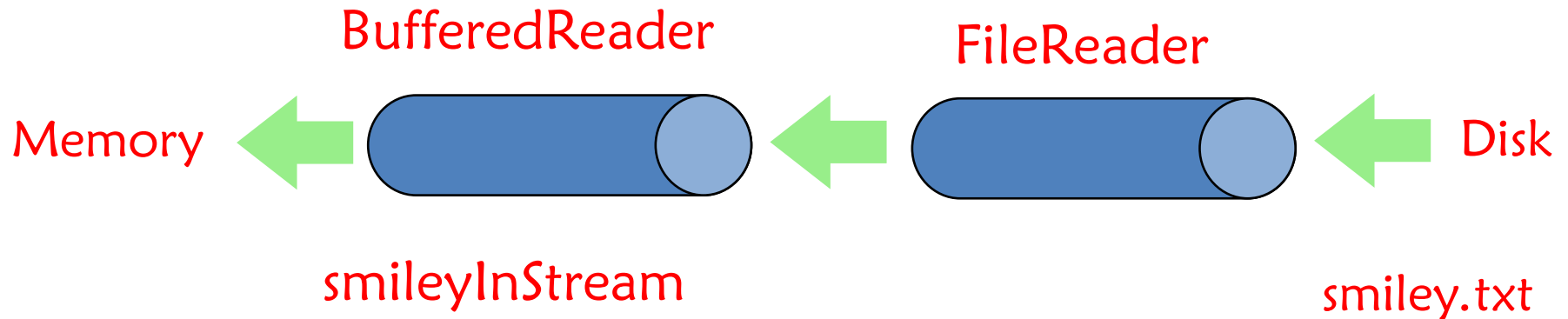
Text File Input

- To open a text file for input: connect a text file to a stream for reading
 - Goal: a **BufferedReader** object,
 - which uses **FileReader** to open a text file
 - **FileReader** “connects” **BufferedReader** to the text file
- For example:

```
BufferedReader smileyInStream = new BufferedReader(new  
    FileReader("smiley.txt"));
```
- Similarly, the long way:

```
FileReader s = new FileReader("smiley.txt");  
BufferedReader smileyInStream = new  
    BufferedReader(s);
```

Input File Streams



```
BufferedReader smileyInStream = new  
    BufferedReader(new  
        FileReader("smiley.txt")) ;
```

Methods for BufferedReader

- **readLine**: read a line into a **String**
- No methods to read numbers directly, so read numbers as Strings and then convert them
- **read**: read a char at a time
- **close**: close BufferedReader stream

Exception Handling with File I/O

Catching IOExceptions

- IOException is a predefined class
- File I/O might throw an IOException
- catch the exception in a catch block that at least prints an error message and ends the program
- FileNotFoundException is derived from IOException
 - therefore any catch block that catches IOExceptions also catches FileNotFoundExceptions
 - put the more specific one first (the derived one) so it catches specifically file-not-found exceptions
 - then you will know that an I/O error is something other than file-not-found

Example: Reading a File Name from the Keyboard

```
public static void main(String[] args)
{
    String fileName = null; // outside try block, can be used in catch
    try
    {
        Scanner keyboard = new Scanner(System.in);
        System.out.println("Enter file name:");
        fileName = keyboard.next();
        BufferedReader inputStream =
            new BufferedReader(new FileReader(fileName));
        String line = null;
        line = inputStream.readLine();
        System.out.println("The first line in " + filename + " is:");
        System.out.println(line);
        // . . . code for reading second line not shown here . . .
        inputStream.close();
    }
    catch(FileNotFoundException e)
    {
        System.out.println("File " + filename + " not found.");
    }
    catch(IOException e)
    {
        System.out.println("Error reading from file " + fileName);
    }
}
```

Testing for End of File in a Text File

- When `readLine` tries to read beyond the end of a text file it returns the special value *null*
 - so you can test for null to stop processing a text file
- `read` returns -1 when it tries to read beyond the end of a text file
 - the int value of all ordinary characters is nonnegative
- Neither of these two methods (`read` and `readLine`) will throw an `EOFException`.

Example: Using Null to Test for End-of-File in a Text File

```
int count = 0;
String line = inputStream.readLine();
while (line != null)
{
    count++;
    outputStream.println(count + " " + line);
    line = inputStream.readLine();
}
```

- When using `read` test for `-1`

Using Path Names

- *Path name* — gives name of file and tells which directory the file is in
- *Relative path name* — gives the path starting with the directory that the program is in
- Typical UNIX path name:
`/user/smith/home.work/java/FileClassDemo.java`
- Typical Windows path name:
`D:\Work\Java\Programs\FileClassDemo.java`
- When a backslash is used in a quoted string it must be written as two backslashes since backslash is the escape character:
`"D:\\Work\\Java\\Programs\\FileClassDemo.java"`
- Java will accept path names in UNIX or Windows format, regardless of which operating system it is actually running on.

File Class [java.io]

- Acts like a wrapper class for file names
- A file name like "numbers.txt" has only String properties
- File has some very useful methods
 - exists: tests if a file already exists
 - canRead: tests if the OS will let you read a file
 - canWrite: tests if the OS will let you write to a file
 - delete: deletes the file, returns true if successful
 - length: returns the number of bytes in the file
 - getName: returns file name, excluding the preceding path
 - getPath: returns the path name — the full name

```
File numFile = new File("numbers.txt");  
if (numFile.exists())  
    System.out.println(numfile.length());
```

Alternative with Scanner

- Instead of BufferedReader with FileReader, then StringTokenizer
- Use Scanner with File:

```
Scanner inFile = new Scanner(new  
    File("in.txt"));
```

- Similar to Scanner with System.in:

```
Scanner keyboard = new  
Scanner(System.in);
```

- **Reading in int's**

```
Scanner inFile = new Scanner(new File("in.txt"));
```

```
int number;
```

```
while (inFile.hasNext())
```

```
{
```

```
    number = inFile.nextInt();
```

```
    // ...
```

```
}
```

- **Reading in lines of characters**

```
Scanner inFile = new Scanner(new  
    File("in.txt"));
```

```
String line;
```

```
while (inFile.hasNextLine())  
{  
    line = inFile.nextLine();  
    // ...  
}
```

Multiple types on one line

```
// Name, id, balance
Scanner inFile = new Scanner(new File("in.txt"));
while (inFile.hasNext())
{
    name = inFile.next();
    id = inFile.nextInt();
    balance = inFile.nextFloat();
    // ... new Account(name, id, balance);
}
```

```
String line;
while (inFile.hasNextLine())
{
    line = inFile.nextLine();
    Scanner parseLine = new Scanner(line) //Scanner again!
    name = parseLine.next();
    id = parseLine.nextInt();
    balance = parseLine.nextFloat();
    // ... new Account(name, id, balance);
}
```

Multiple types on one line

```
// Name, id, balance
Scanner inFile = new Scanner(new File("in.txt"));
String line;
while (inFile.hasNextLine())
{
    line = inFile.nextLine();
    Account account = new Account(line);
}
```

```
-----
public Account(String line) // constructor
{
    Scanner accountLine = new Scanner(line);
    _name = accountLine.next();
    _id = accountLine.nextInt();
    _balance = accountLine.nextFloat();
}
```

BufferedReader vs Scanner (parsing primitive types)

- Scanner
 - `nextInt()`, `nextFloat()`, ... for parsing types
- BufferedReader
 - `read()`, `readLine()`, ... none for parsing types
 - needs `StringTokenizer` then wrapper class methods like `Integer.parseInt(token)`

BufferedReader vs Scanner

(Checking End of File/Stream (EOF))

- **BufferedReader**
 - `readLine()` returns null
 - `read()` returns -1
- **Scanner**
 - `nextLine()` throws exception
 - needs `hasNextLine()` to check first
 - `nextInt()`, `hasNextInt()`, ...


```
BufferedReader inFile = ...  
line = inFile.readLine();  
while (line != null)  
{  
    // ...  
    line = inFile.readLine();  
}
```

```
Scanner inFile = ...  
while (inFile.hasNextLine())  
{  
    line = inFile.nextLine();  
    // ...  
}
```

```
BufferedReader inFile = ...  
line = inFile.readLine();  
while (line != null)  
{  
    // ...  
    line = inFile.readLine();  
}
```

```
BufferedReader inFile = ...  
while ((line = inFile.readLine()) != null)  
{  
    // ...  
}
```

Basic Binary File I/O

- Important classes for binary file **output** (to the file)
 - **ObjectOutputStream**
 - **FileOutputStream**
- Important classes for binary file **input** (from the file):
 - **ObjectInputStream**
 - **FileInputStream**
- Note that **FileOutputStream** and **FileInputStream** are used only for their constructors, which can take file names as arguments.
 - **ObjectOutputStream** and **ObjectInputStream** cannot take file names as arguments for their constructors.
- To use these classes your program needs a line like the following:

```
import java.io.*;
```

Java File I/O: Stream Classes

- **ObjectInputStream and ObjectOutputStream:**
 - have methods to either read or write data one byte at a time
 - automatically convert numbers and characters into binary
 - binary-encoded numeric files (files with numbers) are not readable by a text editor, but store data more efficiently

When Using ObjectOutputStream to Output Data to Files:

- The output files are binary and can store any of the primitive data types (int, char, double, etc.) and the String type
- The files created can be read by other Java programs but are not printable
- The Java I/O library must be imported by including the line:
`import java.io.*;`
 - it contains ObjectOutputStream and other useful class definitions
- An IOException might be thrown

Opening a New Output File

- The file name is given as a **String**
 - file name rules are determined by your operating system
- Opening an output file takes two steps
 1. Create a **FileOutputStream** object associated with the file name String
 2. Connect the **FileOutputStream** to an **ObjectOutputStream** object

This can be done in one line of code

Example: Opening an Output File

To open a file named numbers.dat:

```
ObjectOutputStream outputStream = new  
    ObjectOutputStream(new  
        FileOutputStream("numbers.dat"));
```

- The constructor for `ObjectOutputStream` requires a `FileOutputStream` argument
- The constructor for `FileOutputStream` requires a `String` argument
 - the `String` argument is the output file name
- The following two statements are equivalent to the single statement above:

```
FileOutputStream middleman = new  
    FileOutputStream("numbers.dat");  
ObjectOutputStream outputStream = new  
    ObjectOutputStream(middleman);
```

Some ObjectOutputStream Methods

- You can write data to an output file after it is connected to a stream class
 - Use methods defined in ObjectOutputStream
 - writeInt(int n)
 - writeDouble(double x)
 - writeBoolean(boolean b)
 - etc.
- Note that each write method throws IOException
 - eventually we will have to write a catch block for it
- Also note that each write method includes the modifier final
 - final methods cannot be redefined in derived classes

Closing a File

- An Output file should be closed when you are done writing to it
- Use the close method of the class ObjectOutputStream
- For example, to close the file opened in the previous example:

```
outputStream.close();
```
- If a program ends normally it will close any files that are open

Writing a Character to a File: an Unexpected Little Complexity

- The method `writeChar` has an annoying property:
 - it takes an `int`, not a `char`, argument
- But it is easy to fix:
 - just cast the character to an `int`
- For example, to write the character 'A' to the file opened previously:

```
outputStream.writeChar((int) 'A');
```
- Or, just use the automatic conversion from `char` to `int`

Writing a boolean Value to a File

- `boolean` values can be either of two values, `true` or `false`
- `true` and `false` are not just names for the values, they actually are of type `boolean`
- For example, to write the `boolean` value `false` to the output file:

```
outputStream.writeBoolean(false);
```

Writing Strings to a File:

Another Little Unexpected Complexity

- Use the `writeUTF` method to output a value of type `String`
 - there is no `writeString` method
- UTF stands for Unicode Text Format
 - a special version of Unicode
- UTF is a modification of Unicode that uses just one byte for ASCII characters
 - allows other languages without sacrificing efficiency for ASCII files

When Using `ObjectInputStream` to Read Data from Files:

- Input files are binary and contain any of the primitive data types (`int`, `char`, `double`, etc.) and the `String` type
- The files can be read by Java programs but are not printable
- The Java I/O library must be imported including the line:

```
import java.io.*;
```

 - it contains `ObjectInputStream` and other useful class definitions
- An `IOException` might be thrown

Opening a New Input File

- Similar to opening an output file, but replace "output" with "input"
- The file name is given as a `String`
 - file name rules are determined by your operating system
- Opening a file takes two steps
 1. Creating a `FileInputStream` object associated with the file name `String`
 2. Connecting the `FileInputStream` to an `ObjectInputStream` object
- 2. This can be done in one line of code

Example: Opening an Input File

To open a file named `numbers.dat`:

```
ObjectInputStream inStream = new ObjectInputStream  
    (new FileInputStream("numbers.dat"));
```

- The constructor for `ObjectInputStream` requires a `FileInputStream` argument
- The constructor for `FileInputStream` requires a `String` argument
 - the `String` argument is the input file name
- The following two statements are equivalent to the statement at the top of this slide:

```
FileInputStream middleman = new  
    FileInputStream("numbers.dat");  
ObjectInputStream inputStream = new  
    ObjectInputStream (middleman);
```

Some `ObjectInputStream` Methods

- For every output file method there is a corresponding input file method
- You can read data from an input file after it is connected to a stream class
 - Use methods defined in `ObjectInputStream`
 - `readInt()`
 - `readDouble()`
 - `readBoolean()`
 - etc.
- Note that each write method throws `IOException`
- Also note that each write method includes the modifier `final`

Serialization

- is a mechanism of *writing the state of an object into a byte stream*.
- The reverse operation of serialization is called *deserialization*.
- The String class and all the wrapper classes implements *java.io.Serializable* interface by default.
- **java.io.Serializable interface**
 - Serializable is a marker interface (has no body). It is just used to "mark" java classes which support a certain capability.
 - It must be implemented by the class whose object you want to persist.

Example

```
import java.io.Serializable;
public class Student implements Serializable
{
    int id;
    String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

- **ObjectOutputStream class**

- is used to write primitive data types and Java objects to an OutputStream.
- Only objects that support the `java.io.Serializable` interface can be written to streams.

- **Constructor**

- `public ObjectOutputStream(OutputStream out)` throws `IOException` {}
 - creates an `ObjectOutputStream` that writes to the specified `OutputStream`.

- **Important Methods**

– Method	Description
1) public final void writeObject(specified Object obj) throws IOException {} to the ObjectOutputStream.	writes the object
2) public void flush() throws IOException {}	flushes the current output stream.
3) public void close() throws IOException {}	closes the current output stream.

Example of Java Serialization

- In this example, we are going to serialize the object of Student class.
- The writeObject() method of ObjectOutputStream class provides the functionality to serialize the object.
- We are saving the state of the object in the file named f.txt.

```
import java.io.*;

class Persist{

    public static void main(String args[])throws Exception{
        Student s1 =new Student(211,"ravi");

        FileOutputStream fout=new FileOutputStream("f.txt");

        ObjectOutputStream out=new ObjectOutputStream(fout);
        out.writeObject(s1);
        out.flush();
        System.out.println("success");
    }
}
```

Deserialization in java

- is the process of reconstructing the object from the serialized state. It is the reverse operation of serialization.
- **ObjectInputStream class**
 - deserializes objects and primitive data written using an ObjectOutputStream.
- **Constructor**
 - **public ObjectInputStream(InputStream in) throws IOException {}**
 - creates an ObjectInputStream that reads from the specified InputStream.

- **Important Methods**

- Method

- Description

- 1) `public final Object readObject()`
throws `IOException`,
`ClassNotFoundException` {}

- reads an object from the
input stream.

- 2) `public void close()`
throws `IOException` {}

- closes `ObjectInputStream`.

Example of Java Deserialization

```
import java.io.*;
class Depersist{
    public static void main(String args[])

    throws Exception{
        ObjectInputStream in = new
ObjectInputStream(new
FileInputStream("f.txt"));
        Student s=(Student)in.readObject();
        System.out.println(s.id+" "+s.name);
        in.close();
    }
}
```


Java Serialization with Inheritance (IS-A Relationship)

- If a class implements serializable then all its sub classes will also be serializable.
- Example:

```
import java.io.Serializable;
class Person implements Serializable{
    int id;
    String name;
    Person(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

```
class Student extends Person{
    String course;
    int fee;
    public Student(int i, String n, String c, int f
    {
        super(i,n) ;
        course=c;
        fee=f;
    }
}
```

- you can serialize the Student class object that extends the Person class which is Serializable.
- Parent class properties are inherited to subclasses so if parent class is Serializable, subclass would also be.

Java Serialization with static data member

- If there is any static data member in a class, it will not be serialized because static is the part of class not object.
- Example:

```
class Employee implements Serializable{
    int id;
    String name;
    static String company="SSS IT Pvt Ltd";
        //it won't be serialized
    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Java Transient Keyword

- If you don't want to serialize any data member of a class, you can mark it as transient.
- **Example**
- In this example, we have created the two classes Student and PersistExample. The age data member of the Student class is declared as transient, its value will not be serialized.
- If you deserialize the object, you will get the default value for transient variable.

```
import java.io.Serializable;
public class Student implements Serializable{
    int id;
    String name;
    transient int age;//Now it will not be serialized
    public Student(int id, String name,int age) {
        this.id = id;
        this.name = name;
        this.age=age;
    }
}
```

- the code to serialize the object.

```
import java.io.*;
class PersistExample{
    public static void main(String args[])throws
                                                Exception{
        Student s1 = new Student(211,"ravi",22);
        FileOutputStream f=new FileOutputStream("f.txt");Object
        tOutputStream out=new ObjectOutputStream(f);
        out.writeObject(s1);
        out.flush();
        out.close();
        f.close();
        System.out.println("success");
    }
}
```

Output: success

- Now write the code for deserialization.

```
import java.io.*;
class DePersist{
    public static void main(String args[]) throws
        Exception{
        ObjectInputStream in = new
        ObjectInputStream(new
        FileInputStream("f.txt"));
        Student s =(Student)in.readObject();
        System.out.println(s.id+" "+s.name+" "+s.age);in.close
        ();
    }
}
```

Output: 211 ravi 0

- As you can see, printing age of the student returns 0 because value of age was not serialized.